Lab Walkthrough - Digital Twin: Predictive Insights

Executive Summary: Predictive Digital Twin

The "What"

This lab implements a **Predictive Digital Twin** for modern network infrastructure. It creates an automated data pipeline that feeds real-time telemetry from Cisco Nexus switches, ThousandEyes path intelligence, and Isovalent security flows into a local AI engine (Ollama).

The "Why" (Business Value)

1. **Reactive to Proactive Operations**: Instead of responding to outages after they occur (costing an average of **$9,000 per minute** in enterprise environments), this system identifies "pre-failure" patterns to prevent downtime before it impacts the business.
2. **Data Sovereignty & Security**: By leveraging **local LLMs**, sensitive network topology and security data never leave your infrastructure, eliminating the privacy and compliance risks associated with public AI clouds.
3. **95%+ Inference Cost Savings**:
   - **Cloud AI (e.g., OpenAI/Anthropic)**: Typically costs **$10–$30 per million tokens**. Continuous, high-frequency network monitoring would generate millions of tokens daily, leading to prohibitively expensive monthly invoices.
   - **Local AI (Ollama)**: Running models like **Llama3** locally has **near-zero marginal cost per token**. Once the initial hardware/electricity is accounted for, you can run detailed analysis 24/7 without additional per-message fees.

Technical Components

- **Cisco Nexus 9000**: Simulated telemetry (CPU, Memory, Interface health).
- **Cisco ThousandEyes**: Simulated path intelligence (Latency, Jitter, Loss).
- **Isovalent (Cilium/Hubble)**: Simulated eBPF network flows.
- **Splunk Machine Data Lake**: Centralized storage integration with AWS S3.
- **Ollama**: Local LLM for generating predictive insights.

Logical Topology

```
graph LR
    subgraph Sources
        NX[Nexus 9000]
        TE[ThousandEyes]
        ISO[Isovalent]
    end

    Sources --> SPL[Splunk Enterprise]
    SPL <--> S3[(AWS S3 Data Lake)]
```

```
    subgraph "AI Engine"
        PE[Predictive Engine] <--> OLL[Ollama]
        AID[AI Defense] -.-> OLL
    end

    SPL --> PE
    PE --> AIC[AI Canvas]
```

Setup & Execution

1. Start the Environment

Run the following command to start Splunk, Ollama, and the telemetry producer:

```
docker-compose up -d
```

2. Verify Data Ingestion

1. Access Splunk at `http://localhost:8000`.
2. Login with `admin / ChangedPassword123`.
3. Search for telemetry: `index=* sourcetype="cisco:nexus:9000:telemetry"`

3. Generate Predictive Insights

Execute the predictive engine script:

```
python predictive_engine.py
```

4. GenAI Observability

The lab includes integration with the **GenAI Observability for Splunk** app (App ID: 7382), providing real-time visibility into LLM performance:

- **Token Usage**: Track input/output tokens using OTel semantic conventions.
- **Latency Monitoring**: Measure the duration of AI inference calls.
- **Cost Analysis**: Attribute costs to specific models and workflows.

For detailed installation steps, refer to: genai_observability_setup.md

---

Lab Lifecycle Management

Stop the Lab

To pause the telemetry production while keeping Splunk running:

```
docker-compose stop telemetry-producer
```

To stop all services entirely:

```
docker-compose stop
```

Start/Restart the Lab

To resume from a stopped state:

```
docker-compose start
```

Full Reset (Cold Start)

If you want to clear all data and start fresh:

```
docker-compose down -v
docker-compose up -d
```

> [!NOTE] **Mac ARM Compatibility**: The `splunk` service uses `platform: linux/amd64` in `docker-compose.yml` to ensure it runs correctly on Apple Silicon via Rosetta 2.

Sample Output (Ollama)

> **Predictive Insight Report Anomalies Detected**: CPU usage on Nexus-9000-01 has increased from 75% to 82% over the last 5 minutes. Interface eth1/1 is reporting a rising error count. **Failure Forecast**: 85% probability of interface flapping or localized control plane congestion within the next 30 minutes. **Recommendations**:
>
> 1. Check for routing protocol instability on eth1/1.
> 2. Scale back non-critical telemetry streams to reduce CPU load.
> 3. Verify eBPF flow verdicts from Isovalent to ensure security policies aren't causing unnecessary drops.

Verification

- ✅ Containers running (Splunk, Ollama, Producer)
- ✅ Nexus 9000 telemetry visible in Splunk
- ✅ Ollama generating failure forecasts based on data